



8

10

1800 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

I/O

د. غاندي هسام



برمجة ٣

RB Informatics; 27/11/2025

Using Java for I/O (input/output)

- نظام الإدخال والإخراج (I/O) في جافا مبني على مفهوم Streams أي التدفقات و يُعد تدفق الإدخال/الإخراج مجرد تجريد (Abstraction) يقوم إما بإنتاج المعلومات أو استهلاكها و لا يهم ما نوع الجهاز الذي يتم تبادل البيانات معه ، التركيز يكون على عملية تدفق البيانات أما من البرنامج الى الخارج (Output Stream)، أو من الخارج الى البرنامج (Input Stream)
- يتم ربط ال Stream بجهاز مادي (Physical Device) من خلال نظام الإدخال والإخراج في جافا وجميع هذه ال Streams تتصرف بالطريقة نفسها بغض النظر عن نوع الأجهزة المادية المرتبطة فيها (شاشة الكونسول \ على قرص \ لوحة المفاتيح \ شبكة الانترنت) وبالتالي يمكن تطبيق نفس الأنصاف (Classes) والطرائق (Methods) الخاصة بالإدخال والإخراج على أنواع مختلفة من الأجهزة.

■ ملاحظة :

تتفّذ جافا تدفقات الإدخال والإخراج ضمن تسلسل هرمي من الأنصاف (Class Hierarchies) .

■ : The Byte Stream Classes

■ جافا تنظّم Byte Stream ضمن هرم يبدأ بـ كلاس رئيسيين:

■ : InputStream

■ يمثل المصدر الذي يرسل بيانات إلى برنامجك (قراءة).

■ : OutputStream

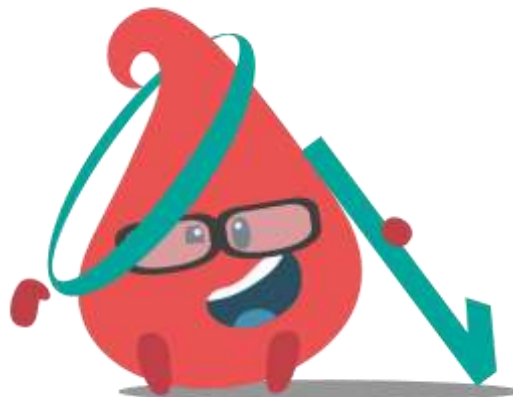
■ يمثل الوجهة التي يرسل إليها برنامجك البيانات (كتابة).

■ كل Byte Stream Classes في جافا ترث من هذين الكلاسين، وتضيف وظائف خاصة حسب نوع الاستخدام.



Byte Stream Class	Meaning
ByteArrayInputStream	تدفق إدخال يقرأ البيانات من مصفوفة بايتات.
ByteArrayOutputStream	تدفق إخراج يكتب البيانات إلى مصفوفة بايتات.
DataInputStream	تدفق إدخال يحتوي على طرائق لقراءة أنواع البيانات القياسية في جافا.
DataOutputStream	تدفق إخراج يحتوي على طرائق لكتابة أنواع البيانات القياسية في جافا.
FileInputStream	تدفق إدخال يقرأ البيانات من ملف.
FileOutputStream	تدفق إخراج يكتب البيانات إلى ملف.
FilterInputStream	صنف وسيط (wrapper) يورث InputStream ويضيف "فلتر/معالجة" على البيانات المقروءة.
FilterOutputStream	يورث OutputStream ويضيف معالجة على البيانات المكتوبة.
InputStream	صنف مجرد (Abstract Class) كل الأصناف التي تتعامل مع الإدخال ترث منه.
OutputStream	صنف مجرد كل الأصناف التي تتعامل مع الإخراج ترث منه.
ObjectInputStream	تدفق إدخال خاص بقراءة الكائنات (Objects).
ObjectOutputStream	تدفق إخراج خاص بكتابة الكائنات.

■ الجدول السابق يوضح مجموعة من الأصناف الأساسية في نظام الإدخال والإخراج في جافا، وجميعها تتعامل مع البيانات على مستوى البايت (Byte).



جدول يوضح بعض الMethod في InputStream :

جميع هذه الMethod هي جزء من آلية قراءة البيانات على مستوى البايت.

Method	Description
int available()	يعيد عدد البايتات المتوفرة حاليًا للقراءة دون الحاجة إلى الانتظار (يعني انتظار وصول بيانات جديدة من مصدر قد يكون بطيئًا أو غير فوري)
Void close()	يغلق مصدر الإدخال. أي محاولة قراءة بعد الإغلاق ستؤدي إلى استثناء من نوع IOException.
int read()	تقرأ بايتًا واحدًا فقط من مصدر الإدخال. القيمة التي تعاد تكون بين 0 و 255 (قيمة البايت)، أو -1 إذا وصلت لنهاية البيانات
int read(byte[] buffer)	تحاول ملء المصفوفة buffer بالبيانات. تقرأ عددًا من البايتات يصل إلى buffer.length و تعيد عدد البايتات الفعلي الذي تمت قراءته. يعيد -1 عند محاولة القراءة بعد نهاية التدفق
int read(byte[] buffer, int offset, int numBytes)	يحاول قراءة ما يصل إلى numBytes بايت في المصفوفة buffer بدءًا من موقع (index) = offset يعيد عدد البايتات المقروءة بنجاح، أو -1 عند الوصول إلى نهاية التدفق.

جدول يوضح بعض الMethod في OutputStream :

جميع هذه الMethod هي جزء من آلية التعامل مع البيانات على مستوى البايت.

Method	Description
Void close()	تغلق تدفق الإخراج. أي محاولة للكتابة بعد ذلك ستؤدي إلى ظهور استثناء من نوع IOException.
void flush()	ترسل أي بيانات مخزنة مؤقتًا إلى وجهتها النهائية. أي أنها تُفرغ المخزن المؤقت للإخراج.

Void write(int b)	تكتب بايت واحد إلى تدفق الإخراج. يُلاحظ أن الوسيط هو من نوع int، مما يسمح باستخدام تعبيرات حسابية دون الحاجة إلى تحويلها إلى byte.
Void write(byte[] buffer)	تكتب مصفوفة كاملة من البايتات إلى تدفق الإخراج أي تستخدم لكتابة مجموعة كاملة من البيانات دفعة واحدة، مما يُحسن الأداء مقارنة بكتابة كل بايت على حدة.
void write(byte[] buffer, int offset)	تكتب جزءاً محدداً من المصفوفة buffer، يبدأ من الفهرس offset و يشمل numBytes من البايتات أي أنها تتيح كتابة جزء معين من البيانات، مما يوفر مرونة في التعامل مع المصفوفات، مثل إرسال جزء من ملف أو بيانات جزئية.

إضافة أريسيزية:

- flush() يفرض إرسال البيانات المتراكمة في ال buffer إلى الوجهة بدون إغلاق المورد.
- close() يغلق ال stream ويحرر المورد، وغالباً يقوم ضمناً ب flush إن وُجد. buffer
- قاعدة: إذا كنت تكتب على ملف/شبكة وتريد التأكد أن البيانات خرجت الآن (قبل الإغلاق)، استخدم flush.

Reading and Writing Files Using Byte Streams:

- الملفات في ال java يتم التعامل معها على مستوى البايت، أي أن البيانات تُقرأ وتُكتب كمتابعة من البايتات. هذا يجعل Java مناسبة للتعامل مع الملفات النصية والملفات الثنائية (مثل الصور والفيديوهات).
- وبما أن معظم العمليات على الملفات تحتاج إلى قراءة أو كتابة بيانات خام، فإن استخدام تدفقات البايتات يُعتبر الأسلوب الأساسي
- ولكن إذا أردنا التعامل مع النصوص (Characters) بدل البايتات مباشرةً، نستخدم كائن وسيط مثل: InputStreamReader يلف (InputStream) ليحوّل البايتات إلى أحرف.
- OutputStreamWriter يلف (OutputStream) ليحوّل الأحرف إلى بايتات قبل كتابتها في الملف.
- لإنشاء تدفق بايت مرتبط بملف، يتم استخدام الفئتين FileInputStream أو FileOutputStream.
- و لفتح ملف، يكفي إنشاء كائن من إحدى هذه الفئات مع تمرير اسم الملف كوسيط (argument) إلى الباني (constructor).

إضافة أريسيزية

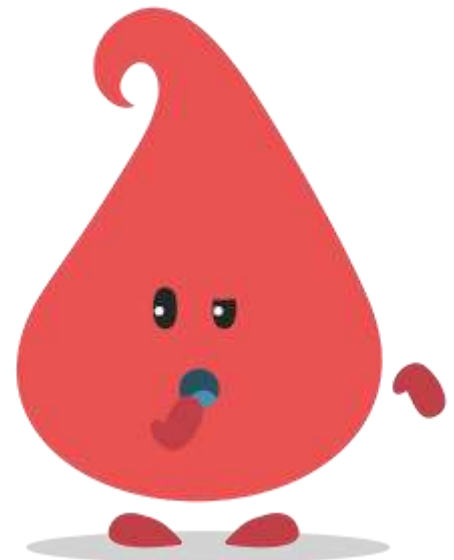
- القراءة/الكتابة "بايت بايت" قد تكون صحيحة وظيفياً لكنها مكلفة؛ لأن كل `read()/write()` قد يسبب **System Call مكلف**
- **Buffering** يقلل عدد مرات الوصول للنظام عبر قراءة/كتابة كتل (Chunks) أكبر ثم التعامل معها داخل الذاكرة.
- **قاعدة عملية:**
- للملفات الكبيرة أو الشبكات: استخدم `BufferedInputStream/BufferedOutputStream` أو `BufferedReader/BufferedWriter`.
- للملفات الصغيرة جداً: قد لا تلاحظ فرقاً، لكن الأفضلية دائماً لصالح `Buffering`.

- القراءة من ملف (Reading from a File)
- عن طريق انشاء كائن من `FileInputStream`، يتم ربط الملف ببرنامجك وفتحه بحيث يمكن قراءة (input) محتواه على شكل بايتات.
- إذا لم يكن الملف موجوداً، يتم رمي استثناء من النوع `FileNotFoundException`.
- للقراءة من الملف، يمكن استخدام الدالة `read()`.
- في كل مرة يتم استدعاؤها، تقوم `read()` بقراءة بايت واحد من الملف وتعيده كقيمة صحيحة (integer).

- **فائدة أريسيزية :**
- تُعيد القيمة كـ `int`، مما يسمح بالتعامل مع القيمة بشكل أوسع من مجرد `byte`
- تُعيد القيمة -1 عند الوصول إلى نهاية الملف.
- تُطلق استثناء من النوع `IOException` عند حدوث خطأ.
- عند الانتهاء من التعامل مع الملف، يجب إغلاقه باستدعاء الدالة `close()`.



```
import java.io.*;
public class ShowFile {
    public static void main(String[] args) {
        int i;
        FileInputStream fin = null;
        try { fin = new FileInputStream("D:\\test.txt");
            do {
                i = fin.read();
                if (i != -1)
                    System.out.print((char) i);
            } while (i != -1);
        } catch (FileNotFoundException exc) {
            System.out.println("File Not Found.");
        } catch (IOException exc) {
            System.out.println("An I/O Error Occurred");
        } finally {
            try {
                if (fin != null) {
                    fin.close();
                }
            } catch (IOException exc) {
                System.out.println("Error Closing File");
            }
        }
    }
}
```



■ إضافة أريسيزية:

استخدمنا try-with-resources لأنه يضمن إغلاق الملف حتى لو صار Exception. وبـ do-while نقرأ أول بايت ثم نكرر حتى ترجع read() القيمة -1

مثال 2:

```

public static void main(String[] args) {
    FileInputStream fin = null;
    try {
        fin = new FileInputStream("D:\\test.txt");
        int size = fin.available();
        byte[] bytes = new byte[size];
        fin.read(bytes);
        System.out.println(new String(bytes));
    }
    catch (Exception exc) {
        System.out.println("An Error Occurred: " + exc);
    } finally {
        try {
            if (fin != null) {
                fin.close();
            }
        } catch (IOException exc) {
            System.out.println("Error Closing File");
        }
    }
}

```



■ الكتابة إلى ملف (Writing to a File) :

■ لفتح ملف من أجل الإخراج (Output) ، نقوم بإنشاء كائن من الفئة `FileOutputStream`

■ هناك نوعين من ال (Constructors) شائعان للاستخدام:

- `FileOutputStream(String fileName)`
- `FileOutputStream(String fileName, boolean append)`

عند استخدام الباني الأول

`FileOutputStream(String fileName)`

يتم إنشاء ملف جديد باسم محدد، وإذا كان هناك ملف موجود بنفس الاسم، فإنه يُستبدل ويُفقد محتواه السابق .
أما الباني الثاني

`FileOutputStream(String fileName, boolean append)`

■ فهو يتيح خيارين:

- إذا كانت القيمة `append = false` ، فإن الملف يُعاد إنشاؤه ويفقد محتواه السابق.
- إذا كانت القيمة `append = true` ، فإن البيانات الجديدة تُضاف في نهاية الملف دون حذف البيانات القديم
- عملية الكتابة تتم عبر استدعاء الدالة `write()` ، والتي تسمح بكتابة البيانات (عادةً على شكل مصفوفة من البايتات) داخل الملف.

```

public class WriteToFile {
    public static void main(String[] args) {
        String str = "Hello There!";
        FileOutputStream fout = null;
        try {
            fout = new FileOutputStream("D:\\test.txt");
            byte[] bytes = str.getBytes();
            fout.write(bytes);
        } catch (Exception exc) {
            System.out.println(exc.getMessage());
        } finally {
            try {
                if (fout != null) {
                    fout.close();
                }
            } catch (IOException exc) {
                System.out.println("Error Closing File");
            }
        }
    }
}

```

الإغلاق التلقائي للملف (Automatically Closing a File) :

في Java ، عند التعامل مع الملفات أو الموارد التي تحتاج إلى إغلاق يدوي (مثل FileInputStream أو BufferedReader)، يمكن استخدام try-with-resources لضمان إغلاق هذه الموارد تلقائياً بمجرد انتهاء الكتلة try

try-with-resources تُترجم داخلياً إلى:

إنشاء المورد

تنفيذ كتلة try

إغلاق المورد في النهاية حتى لو حدث Exception

```
public class ShowFile {
    public static void main(String[] args) {
        try (FileInputStream fin = new
FileInputStream("D:\\test.txt")) {
            int size = fin.available();
            byte[] bytes = new byte[size];
            fin.read(bytes);
            System.out.println(new String(bytes));
        }
        catch (IOException exc) {
            System.out.println("An Error Occurred: " + exc);
        }
    }
}
```

لقراءة وكتابة البيانات الثنائية (Reading and Writing Binary Data)

في أمثلتنا السابقة تعاملنا مع البايتات على أنها محارف بسيطة عند الطباعة، لذلك ظهرت كـ ASCII

- **فائدة أريسيزية:**
- Byte Streams لا تقتصر على ASCII ؛ هي تنقل "بايتات خام" لأي نوع بيانات. الذي جعل الأمر يبدو كـ ASCII هو أننا كنا نحول البايت إلى char عند الطباعة.
- رموز ASCII هي مجموعة من القيم الرقمية التي تمثل الأحرف والرموز في الحاسوب، وتُستخدم لتبادل المعلومات بين الأنظمة الإلكترونية. تشمل الحروف، الأرقام، علامات الترقيم، وأوامر التحكم
- مثال:
- الرمز 'A' في ASCII = العدد 65.
- الرمز '0' في ASCII = العدد 48.
- ولكن في بعض الحالات، قد تحتاج إلى إنشاء ملف يحتوي على أنواع بيانات أخرى مثل int أو double أو short.
- لقراءة وكتابة القيم الثنائية لأنواع البيانات البدائية في Java ، نستخدم الكلاسين:
 - DataInputStream
 - DataOutputStream
- لكتابة البيانات إلى ملف، نمرر كائن من نوع FileOutputStream إلى مُنشئ DataOutputStream
- وللقراءة من ملف، نمرر كائن من نوع FileInputStream إلى كونسرتكر DataInputStream

```
import java.io.*;
public class RWDData {
    public static void main(String[] args) {
        int i = 10;
        double d = 1023.56;
        boolean b = true;
        try (DataOutputStream dataOut
            = new DataOutputStream(new FileOutputStream("testdata"))) {
            dataOut.writeInt(i);
            dataOut.writeDouble(d);
            dataOut.writeBoolean(b);
        } catch (IOException exc) {
            System.out.println("Write error.");
            return;
        }
        // Now, read them back.
        try (DataInputStream dataIn
            = new DataInputStream(new FileInputStream("testdata"))) {
            i = dataIn.readInt();
            System.out.println("Reading " + i);
            d = dataIn.readDouble();
            System.out.println("Reading " + d);
            b = dataIn.readBoolean();
            System.out.println("Reading " + b);
        } catch (IOException exc) {
            System.out.println("Read error.");
        }
    }
}
```



التدفقات المعتمدة على الأحرف (Character Streams) :

تُستخدم لقراءة وكتابة النصوص بدلاً من البيانات الثنائية.

تتعامل مع رموز Unicode ، مما يجعلها مثالية لتخزين النصوص بلغات متعددة مثل العربية، الصينية، إلخ.

تُستخدم الكلاسات `FileReader` و `FileWriter` بدلاً من `FileInputStream` و `FileOutputStream`

■ فائدة أريسيزية :

■ ما هو Unicode ؟

الفكرة الأساسية منه تعيين رقم فريد لكل حرف أو رمز، بغض النظر عن اللغة أو النظام الكتابي مثل :

ليكن `char arabicChar = 'س'`

هذا حرف عربي، داخلياً يُخزن كقيمة. `Unicode = U+0633`

`char latinChar = 'A';`

هذا حرف لاتيني، داخلياً يُخزن كقيمة. `Unicode = U+0041`

`String emoji = "\u1F600";`

هذا الايموجي المبتسم 😊

Character Stream Classes

يتم تعريف تدفقات المحارف في Java باستخدام تسلسلين هرميين من الفئات، يتصدرهما الفتان

المجردتان `Reader` و `Writer`.

■ في بعض الحالات، يمكن أن تكون تدفقات المحارف أكثر كفاءة من تدفقات البايت.

■ التحويل بين `Bytes` و `Chars` يتم عبر `Encoding/Charset`.

■ `FileReader/FileWriter` يستخدمان ترميز النظام الافتراضي (قد يختلف بين `Windows/Linux`).

■ لضمان نفس النتائج على كل الأجهزة، حدّد `UTF-8` صراحة.

فيما يلي بعض الفئات المستخدمة في تدفقات المحارف:

class	Description
CharArrayReader	تدفق إدخال يقرأ من مصفوفة محارف
CharArrayWriter	تدفق إخراج يكتب إلى مصفوفة محارف
FileReader	تدفق إدخال يقرأ من ملف
FileWriter	تدفق إخراج يكتب إلى ملف

بعض الدوال المعرفة في الفئة Reader :

Method	Description
int read(char[] buffer)	تحاول قراءة حتى <code>buffer.length</code> حرفاً داخل المصفوفة <code>buffer</code> ، وتُرجع عدد المحارف المقروءة فعلياً. تُرجع -1 عند الوصول لنهاية التدفق
Abstract int read(char[] buffer, int offset, int numChars)	تحاول قراءة حتى <code>numChars</code> حرفاً داخل <code>buffer</code> بدءاً من الموقع <code>offset</code> تُرجع عدد المحارف المقروءة فعلياً، وتُرجع -1 عند نهاية التدفق.

بعض الدوال المعرفة في الفئة Writer :

Method	Description
writer append(char ch)	يُضيف الحرف <code>ch</code> إلى نهاية تدفق الإخراج المستدعي. تُرجع مرجعاً لنفس التدفق.
Void write(char[] buffer)	تكتب كامل مصفوفة المحارف <code>buffer</code> إلى تدفق الإخراج المستدعي.
Void write(String str)	تكتب السلسلة النصية <code>str</code> إلى تدفق الإخراج المستدعي.
Void write(String str, int offset ,int numChars)	تكتب جزءاً فرعياً من <code>str</code> يحتوي على <code>numChars</code> حرفاً بدءاً من الموقع <code>offset</code>



مثال لكود القراءة من ملف Read Example:

```
import java.io.*;
public class ReadFromFile {
    public static void main(String[] args) {
        int i;
        try (FileReader fr = new FileReader("D:\\test.txt")) {
            String str = "";
            do {
                i = fr.read();
                if(i != -1) str = str + (char) i;
            } while (i != -1);
            System.out.println("x: " + str);
        } catch (IOException exc) {
            System.out.println("I/O Error: " + exc);
        }
    }
}
```

مثال لكود الكتابة إلى ملف Write Example

```
import java.io.*;
public class WriteToFile {
    public static void main(String[] args) {
        try (FileWriter fr = new FileWriter("D:\\test.txt")) {
            String str = "New content";
            fr.write(str);
        } catch (IOException exc) {
            System.out.println("I/O Error: " + exc);
        }
    }
}
```

Byyyyyeee

